

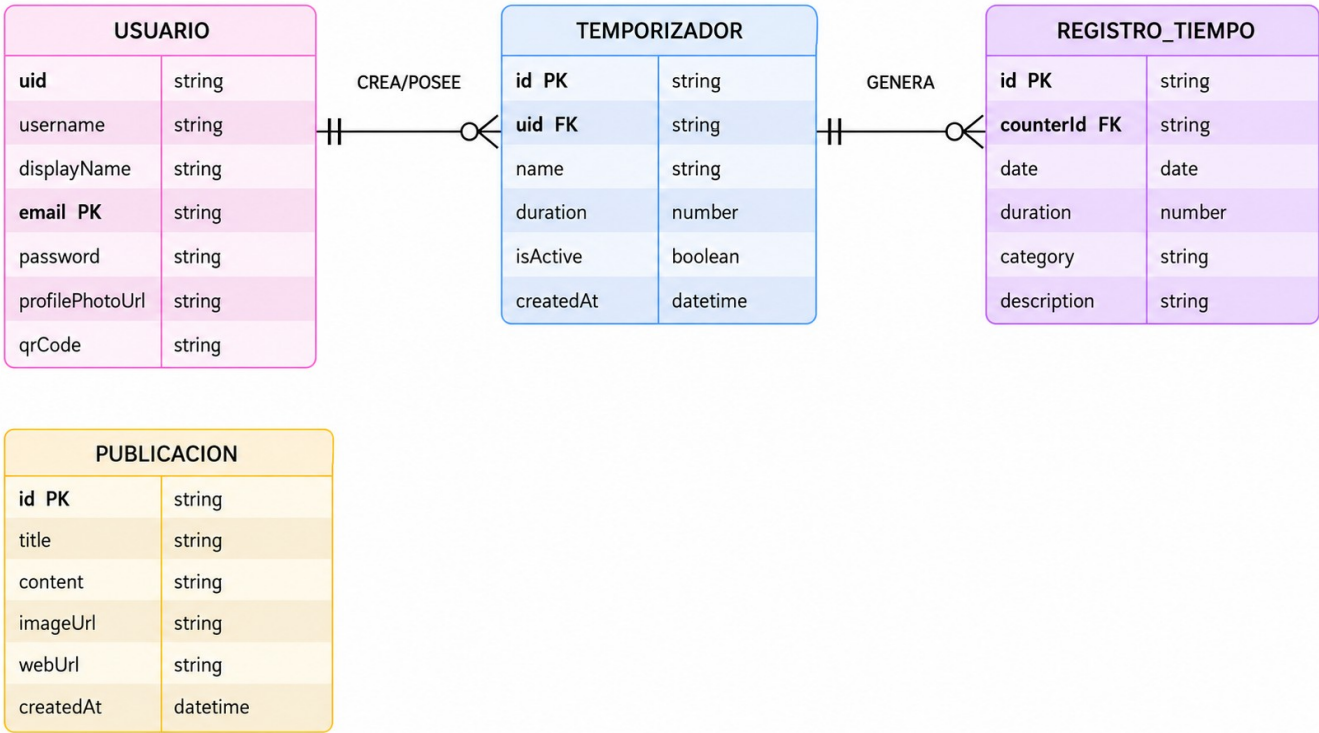
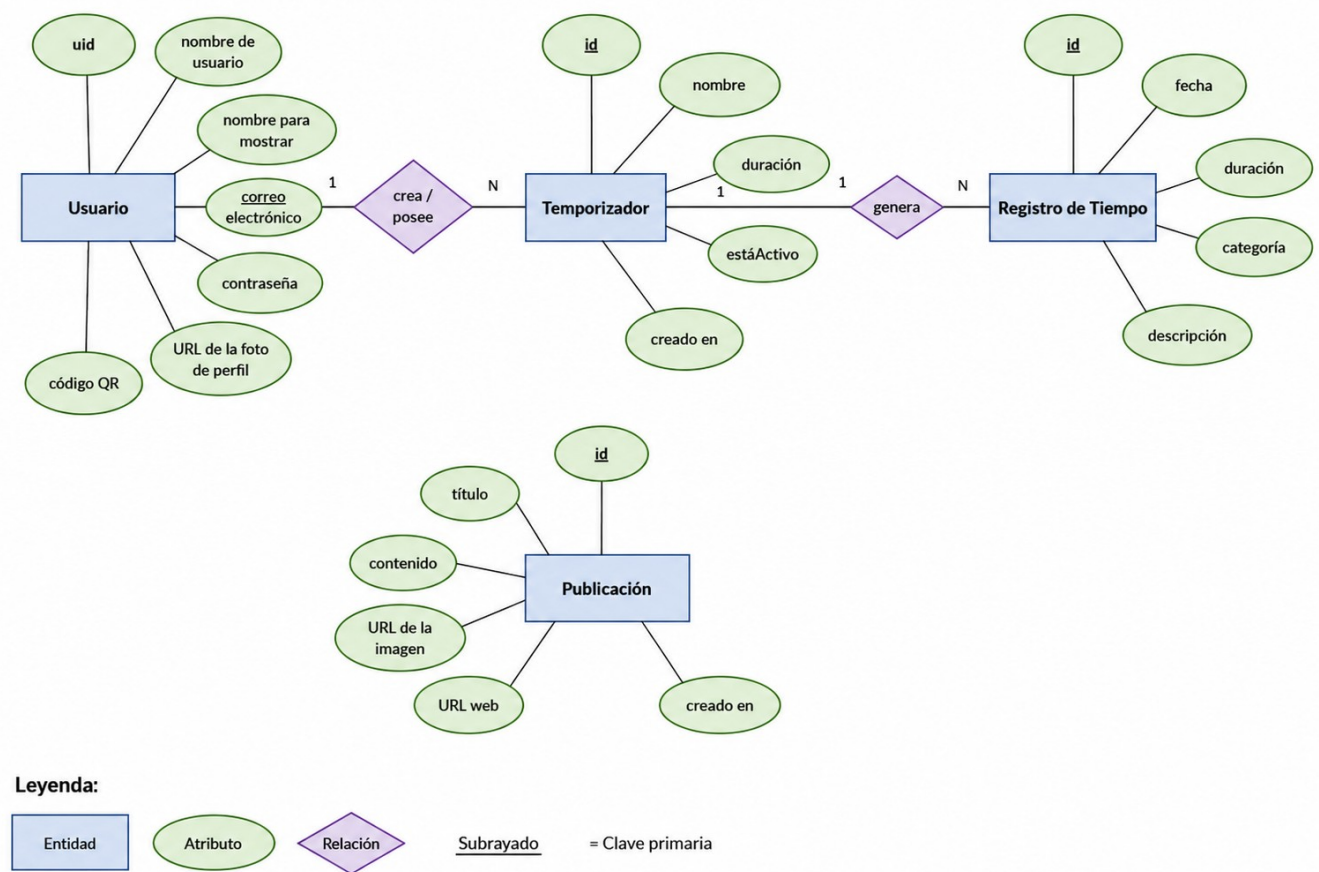


Diagrama Entidad - Relación

Temporis:

Una herramienta
accesible para el
control y la
optimización de
nuestro tiempo

1. Diagrama Entidad - Relación



1.1 Introducción al Modelo de Datos

El presente documento detalla el **Diagrama Entidad-Relación (E-R)** del proyecto **Temporis**. El diseño de este modelo se ha desvinculado de la rigidez de los ficheros de código fuente para centrarse en la estructura lógica de los datos que persisten en el *backend*. Cada entidad responde de manera directa a los requisitos de almacenamiento, análisis de tiempos y distribución de contenidos informativos de la aplicación.

2. Justificación Tecnológica: NoSQL (Firestore) vs. Relacional (SQL)

Para el desarrollo de *Temporis*, se ha optado por implementar una base de datos *NoSQL* orientada a documentos mediante *Firebase Cloud Firestore*, descartando opciones relacionales tradicionales como *MySQL* con *PhpMyAdmin*. Esta decisión se fundamenta en los siguientes pilares estratégicos:

- **Escalabilidad Dinámica:** A diferencia de *MySQL*, donde los esquemas de tablas son rígidos, *Firestore* permite que cada documento contenga una estructura flexible de colecciones. Esto facilita la expansión futura de la aplicación sin necesidad de realizar costosas migraciones de esquemas que comprometan la disponibilidad del servicio en producción.
- **Sincronización en Tiempo Real:** *Firestore* ofrece una integración nativa mediante “oyentes” (*listeners*). Para una aplicación enfocada en la gestión del tiempo como *Temporis*, esto garantiza que si un usuario modifica o detiene un temporizador en un dispositivo, la actualización se propaga instantáneamente a cualquier otra sesión activa del usuario.
- **Gestión de Datos Desconectados (Offline):** Las aplicaciones móviles operan en entornos de conectividad variable. *Firestore* gestiona de forma automática una caché local persistente en el dispositivo, lo que permite que *Temporis* continúe registrando tiempos sin acceso a Internet y sincronice las colecciones de forma transparente al recuperar la cobertura.
- **Optimización de Consultas:** Al no requerir operaciones de combinación complejas (*JOINS*), la velocidad de lectura de documentos se mantiene constante y ligada únicamente al tamaño del conjunto de resultados solicitado, asegurando una experiencia de usuario fluida a medida que escala la plataforma.

3. Seguridad y Control de Acceso: Firebase Security Rules

A diferencia de una arquitectura *SQL* tradicional, donde la seguridad reside exclusivamente en un servidor intermedio (*Backend*), *Firestore* permite acoplar las reglas de validación y autorización directamente sobre la capa de datos mediante las *Security Rules*. Esto garantiza:

- **Privacidad Estricta del Usuario:** Se han implementado reglas basadas en la propiedad `request.auth.uid`. Una petición de lectura o escritura sobre un documento de las colecciones de temporizadores o registros de tiempo solo será autorizada si el campo `uid` del documento coincide con el identificador único del usuario autenticado.
- **Integridad de Tipos en Tiempo Real:** Las reglas actúan como validadores de esquemas, rechazando escrituras que contengan atributos malformados o tipos de datos inconsistentes con las especificaciones del modelo.
- **Principio de Menor Privilegio:** Cualquier intento de acceso que no esté explícitamente permitido por una regla de seguridad es denegado por defecto, blindando los datos de vectores de acceso maliciosos.

4. Descripción de Entidades y Atributos

4.1. Definición de Entidades y Atributos

Para adaptar el modelo al estándar de bases de datos, se corrigen los tipos de datos de programación (*char*, *bool*, *long double*) por tipos de datos de almacenamiento estándar (*String*, *Boolean*, *Integer*, *Timestamp*).

1. USUARIO

Representa a los usuarios registrados en la plataforma a través del módulo de autenticación.

- **uid: String** - Identificador único provisto por *Firebase Auth*. (de gestión interna)
- **username: String** - Nombre de usuario único del perfil.
- **displayName: String** - Nombre público visible del usuario.
- **email (PK): String** - Dirección de correo electrónico de registro.
- **password: String** - *Hash* de seguridad de la contraseña (gestionado por el proveedor de autenticación).
- **profilePhotoUrl: String** - Enlace al almacenamiento de la imagen de perfil.
- **qrCode: String** - Cadena de texto codificada para la generación del código QR de perfil. (Actualmente no se usa, aunque en un futuro podría contribuir al desarrollo de alguna funcionalidad interesante y de utilidad considerable)
- ~~**rol: Integer** - Indicador numérico de privilegios (ej. 0: Usuario estándar, 1: Administrador).~~
(Actualmente no se está usando y la administración de los “posts” y de los “users” se realiza desde el portal web de Firebase. Por ende, se ha decidido eliminarlo del/desconsiderarlo en diagrama ER actual. PD: No se descarta volver a integrarlo en un futuro para poder utilizar ese “token” a la hora de gestionar los criterios de acceso y los privilegios de edición, definidos en su momento para cada usuario/a de nuestra aplicación).

2. TEMPORIZADOR

Entidad que almacena los contadores y configuraciones de tiempo personalizadas.

- **id (PK): String** - Identificador único del documento del temporizador. (de gestión interna)
- **name: String** - Título o etiqueta del temporizador.
- **duration: Integer** - Tiempo total configurado en segundos.
- **isActive: Boolean** - Estado actual del temporizador (Activo / Pausado).
- **createdAt: Timestamp** - Fecha y hora exacta de creación.
- ~~**uid (FK): String** - Clave foránea que vincula el temporizador con su Usuario propietario.~~

3. REGISTRO_TIEMPO

Historial de sesiones de tiempo consolidadas y efectivamente ejecutadas.

- **id (PK): String** - Identificador único del registro de actividad. (de gestión interna)
- **date: Timestamp** - Fecha en la que se realizó la sesión de cronometraje.
- **duration: Integer** - Cantidad de segundos efectivamente transcurridos y grabados.
- **category: String** - Etiqueta de categorización de la actividad (ej. "Estudio", "Trabajo").

- **description: String** - Breve anotación u observación opcional del usuario sobre la sesión.
- ~~**counterId (FK): String** - Clave foránea que conecta el registro con el Temporizador de origen.~~

4. PUBLICACIÓN

Contenido informativo o educativo distribuido de forma global a los usuarios de la plataforma. Es una entidad independiente (sin dependencias foráneas directas en este nivel).

- **id (PK): String** - Identificador único del *post*. (de gestión interna)
- **title: String** - Título descriptivo de la publicación.
- **content: String** - Cuerpo del texto o mensaje.
- **imageUrl: String** - Enlace al recurso gráfico o miniatura del *post*.
- **webUrl: String** - URL externa de referencia para ampliar la información.
- **createdAt: Timestamp** - Fecha de edición o lanzamiento de la publicación.

4.2 Relaciones y Lógica de Negocio (Cardinalidades)

- Usuario - Temporizador (1:N): Un usuario puede dar de alta y gestionar múltiples temporizadores personalizados dentro de su aplicación. Sin embargo, un documento de temporizador pertenece única y exclusivamente a un usuario específico, quedando vinculado mediante el campo **uid**. La cardinalidad es estructuralmente 1 a N.
- Temporizador - Registro de Tiempo (1:N): Un mismo temporizador puede ser ejecutado, guardado y finalizado infinitas veces a lo largo de las semanas, generando un historial ramificado. Por lo tanto, un temporizador puede estar asociado a múltiples registros de tiempo históricos, mientras que cada registro individual está ligado al temporizador que lo accionó a través del atributo **counterId**. La cardinalidad es 1 a N.

5. Notas sobre la Optimización del Modelo

En consonancia con el desarrollo real de la infraestructura del software, la entidad abstracta preliminar **Counter.java** (Contador) ha sido depurada e integrada por completo dentro de la entidad **TEMPORIZADOR**.

Esta unificación conceptual elimina campos redundantes y nulos (como el antiguo **counterid(FK)** de la versión previa), centralizando el flujo de datos (aunque en la clase **TimerRegister.java** sigue existiendo, por si nos resulta nuevamente necesario usarlo en un futuro). De esta manera, cualquier módulo de analítica o registro lee directamente del identificador del temporizador activo, garantizando un modelo normalizado, limpio y optimizado para consultas directas en entornos móviles de alta velocidad.